PORTFOLIO SAMPLE · ELECTRICAL SYSTEMS ENG. INTERN · SUMMER 2027

Ranger TVC: CAN-FD Gimbal Actuation for a VTVL Rocket Vehicle

Adit Bansal
B.S. Electrical & Computer Eng., USC '28
GNC Team · USC Liquid Propulsion Lab
aditbans@usc.edu

Ranger is USC Liquid Propulsion Lab's vertical-takeoff, vertical-landing (VTVL) liquid-rocket vehicle. To steer, the engine sits on a two-axis gimbal pushed by two electric linear actuators (pitch and yaw). I own the **thrust vector control (TVC) system** from the flight computer to the actuator: the CAN-FD bus to the motor controllers, the real-time 50 Hz control thread, the gimbal kinematics, and the safety supervisor that decides when the actuators are allowed to move.

50 Hz deterministic loop, kernel-timer paced (no drift)	**< 5 ms** command-to-reply latency budget per cycle	**±12°** two-axis gimbal, ~2.4 in actuator stroke	**5 faults** detected and latched into a safe state	**55** host unit tests incl. MATLAB-generated vectors

01 System architecture

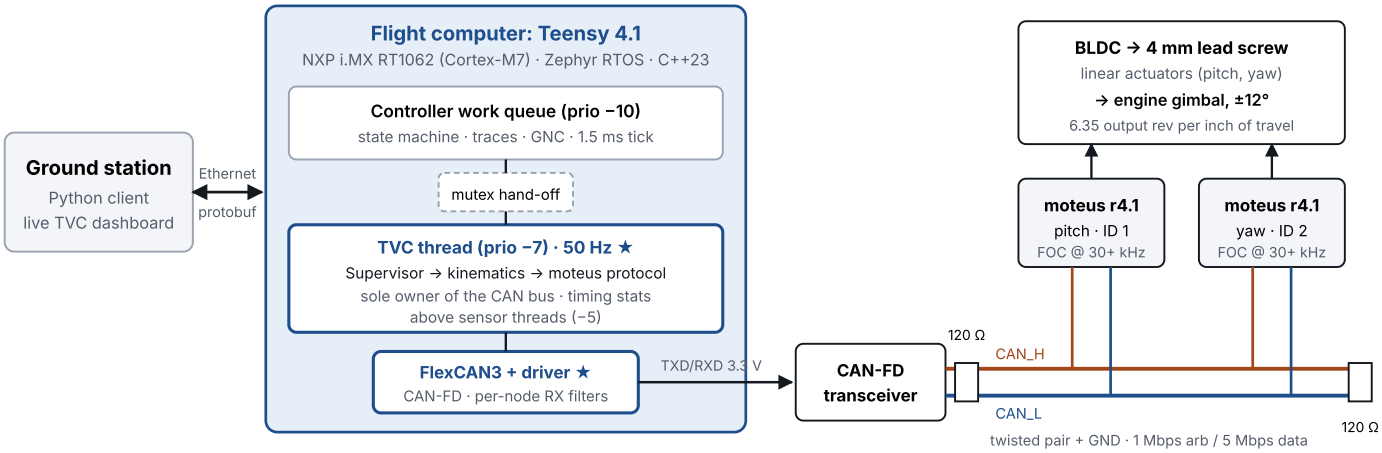


Fig. 1. TVC signal path. ★ = blocks I designed and implemented. moteus closes the motor current and position loops internally; my firmware sends a position setpoint plus limits, and queries the measured state, every 20 ms.

My role and contributions mine

- Brought up **CAN-FD on FlexCAN3**: devicetree overlay, bit timing, Kconfig, pin mapping to the transceiver
- Wrote an **allocation-free moteus protocol** encoder/decoder and a non-blocking CAN transport
- Built the **50 Hz TVC thread** with jitter, latency and overrun telemetry, and resolved thread-timing races
- Implemented **closed-form inverse kinematics** (law of cosines) from the mechanical team's gimbal model
- Designed the **safety supervisor**: homing, enable interlocks, clamps, slew limits, fault latching
- Debugged the **CAN physical layer** with an oscilloscope and SocketCAN tools; wrote the bench procedures

Team context team

The GNC team owns guidance and control software for Ranger. Teammates built the flight-computer controller framework, the sensor drivers, the first moteus bench test (which set the controller IDs and defaults), and the gimbal mechanism and its MATLAB kinematic model, which I translated into firmware.

Tools

Zephyr RTOS · C++23 · devicetree/Kconfig · CAN-FD / SocketCAN · moteus r4.1 · oscilloscope · MATLAB · Python · protobuf (nanopb) · ztest on native_sim · Git

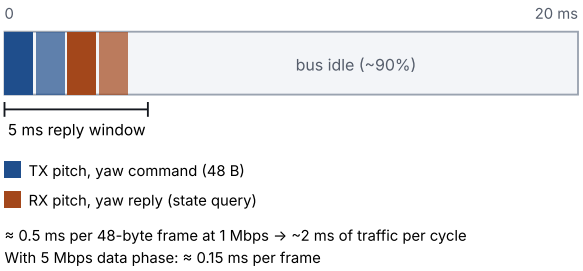
02 Electrical interface: the CAN-FD bus

Why CAN-FD. Each actuator needs a 43-byte command-and-query frame every cycle. Classic CAN carries 8 data bytes per frame, so one command would need six frames. CAN-FD carries up to 64 bytes and can switch the data phase to a higher bit rate. Because CAN is a differential bus, it also tolerates the electrical noise around a firing engine far better than the single-ended PWM servo links used on our earlier test vehicle.

Physical-layer design decisions

Decision	Reason
Transceiver with 3.3 V logic I/O	Teensy 4.x pins are **not 5 V tolerant**; a 5 V-logic transceiver would damage the MCU
120 Ω at **both** bus ends	moteus boards have no on-board termination. The bus impedance is ~120 Ω, so an unterminated end reflects edges and corrupts bits
Twisted CAN_H/CAN_L + shared GND	Noise couples equally into both wires and cancels in the differential receiver; GND keeps both nodes inside the common-mode range
Standby pin tied for normal mode	A floating standby/silent pin leaves the transceiver unable to drive the bus or ACK
1 Mbps arbitration / 5 Mbps data, BRS off at first	Bench-validated at 1 Mbps first; switch to 5 Mbps data only after checking signal quality on the real vehicle harness

Bus timing budget (one 20 ms cycle)



≈ 0.5 ms per 48-byte frame at 1 Mbps → ~2 ms of traffic per cycle
With 5 Mbps data phase: ≈ 0.15 ms per frame

Frame-time estimate

```
t_frame ≈ 67 bit / R_arb + (48 B·8 + CRC + stuffing ≈ 430 bit) / R_data ≈ 0.5 ms at 1 Mbps
```

That gives four frames, about 2 ms, inside a 5 ms reply window, which leaves margin for a retransmission and for bus load to grow when more nodes are added. A 48-byte frame was also a deliberate choice: I encode every position register as **float32** rather than moteus' scaled int16 format, because int16 saturates at ±3.28 revolutions and the actuator travels about ±8 rev over ±12°. Spending 5 extra bytes per frame removes an overflow failure mode entirely.

Firmware ↔ hardware interface

- **Devicetree overlay** enables FlexCAN3 on Teensy pins 31 (CTX3) and 30 (CRX3), sets both bit rates, and selects it as the board's `zephyr,canbus`. The build fails at compile time if CAN-FD is not configured.
- **Hardware acceptance filters** on all 29 bits of the extended ID, one per controller. Each reply lands in its own message queue, and the CPU never sees unrelated traffic.
- **Non-blocking transmit**: frames go into a hardware mailbox with a completion callback, so a missing ACK or a stuck bus cannot stall the control thread.
- **Watchdog on the motor controller**: if commands stop for 100 ms (cable cut, MCU hang), moteus stops driving on its own. The hardware fails safe independently of my code.

Bringing up and debugging the physical layer

Approach: verify the bus one layer at a time, so a "no reply" symptom can be pinned to wiring, signaling, or firmware instead of guessing.

Method: probed CAN_H and CAN_L with an oscilloscope and checked the differential levels (recessive ≈ 0 V, dominant ≈ 2 V). Looked for ringing on edges, which points to termination problems. Measured ~60 Ω across H–L with power off, which confirms both 120 Ω terminators are present. Used SocketCAN (`candump`) to confirm frame IDs and ACKs independently of the MCU.

Firmware side: matched the 29-bit ID layout (prefix, source, destination, reply bit) and bit rates to the moteus configuration, and brought up one actuator at a time with a single-axis bench build, so each fault had only one suspect.

03 Control chain: from gimbal angle to motor revolutions

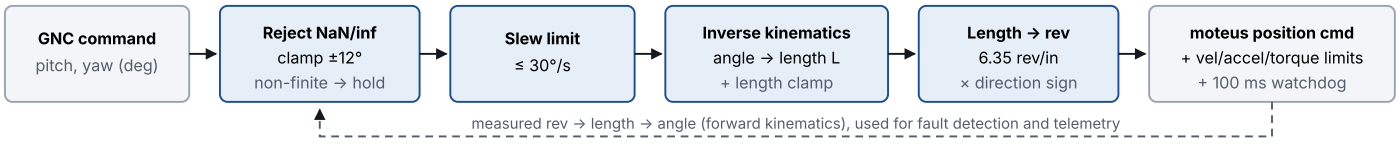
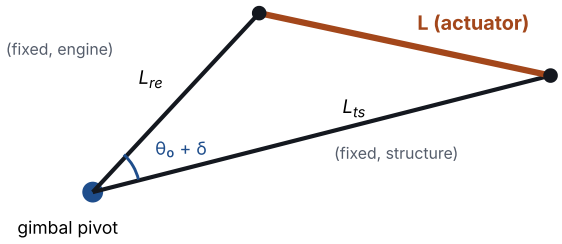


Fig. 2. The command chain runs every 20 ms. Every stage is a pure function, unit-tested on the host before it ever touches hardware.

Inverse kinematics (closed form)



$$L(\delta)^2 = L_{ts}^2 + L_{re}^2 - 2 \cdot L_{ts} \cdot L_{re} \cdot \cos(\theta_o + \delta)$$

δ is the commanded gimbal angle and θ_o is the angle between the fixed arms at center. Only *L* changes, so I precompute the constants at **compile time**. The inverse, $\delta(L) = \text{acos}(\ldots) - \theta_o$, turns measured motor position back into an angle. Across ±12°, $|dL/d\delta| \leq 0.115$ in/deg, which I use to convert angular rate limits into actuator velocity limits conservatively.

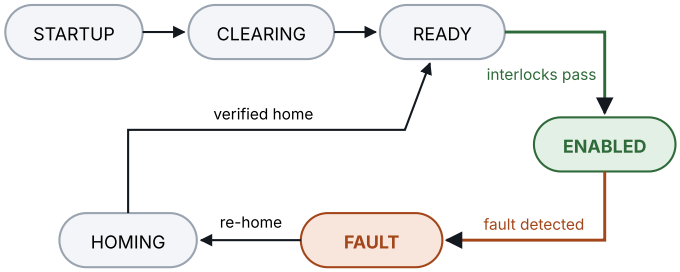
Validated against the mechanical model: 241 test vectors per axis (−12° to +12° in 0.1° steps) from the team's MATLAB model, enforced by unit tests. While building them I noticed the supplied yaw table was **identical to the pitch table**, even though the MATLAB model gives yaw a different center angle (a ~0.35 in difference in center length). I flagged it to the mechanical team instead of guessing, and the yaw axis stays disabled until its geometry and motor direction are confirmed.

Sanity-check numbers

Quantity	Value
Actuator length, −12° → center → +12°	17.08 → 18.36 → 19.47 in
Motor travel over ±12° (6.35 rev/in)	−8.1 to +7.0 rev (asymmetric: cos nonlinearity)
30°/s slew × 0.115 in/deg	≤ 3.45 in/s ≈ 22 rev/s at the motor
Lead: 4 mm single-start, 1:1 drive	25.4 / 4 = 6.35 rev per inch

The ±8 rev travel is why position is sent as float32 rather than int16 (saturates at ±3.28 rev); see p. 2.

Safety supervisor (state machine)



Enable interlocks. The actuators only move when all of these hold: the configuration has no placeholder values, both controllers reply, both report a valid home reference, both are stopped and fault-free, and the measured length is within 0.05 in of center. **Position mode can never be entered on an unhomed actuator.**

Fault (latched)	Detection
No reply	3 consecutive missed cycles (60 ms)
Controller fault	moteus mode = fault; code preserved for debug
Watchdog timeout	moteus reports timeout mode
Wrong mode	not in position mode while enabled
Tracking error	$\lvert\text{cmd} - \text{meas}\rvert > 0.10$ in for 5 cycles

Response: healthy axes slew back to center, faulted axes are stopped, and an active static-fire sequence aborts.

04 Real-time design, verification, and results

Deterministic timing

- **Absolute-period scheduling:** the loop waits on a periodic kernel timer instead of sleeping after its work. Sleeping after the work makes the period 20 ms *plus* execution time, so the loop drifts. Missed periods are counted as overruns rather than silently stretching the cycle.
- **Priority placement:** the TVC thread sits below the main controller tick and above all sensor threads, so a slow sensor read can never delay an actuator command.
- **No priority inversion:** the controller and TVC thread share one small struct behind a mutex. Both only copy data while holding it and never block or log inside it, so the controller never waits on the CAN bus.
- **Self-measuring:** every cycle records period jitter, per-axis reply latency, cycle time and overruns, and streams them to the ground station.

Verification strategy

Level	What it proves
55 host unit tests (ztest, native_sim)	IK vs 241 MATLAB vectors/axis; frame encode/decode byte-exact; every supervisor safety path (missed replies, faults, interlocks, slew, NaN)
Single-actuator bench build	One axis on a bare Teensy + transceiver; the full safety logic still runs on the installed axis
Bench sweep	±2° triangle sweep at 0.5°/s; commanded vs measured angle logged at 5 Hz
Ground dashboard	Live state, enable blockers, per-axis cmd/meas, voltage, temperature, latency; CSV log for plots

Timing budget per 20 ms cycle (design limits, monitored live by built-in telemetry)

< 5 ms	~2 ms	60 ms	100 ms
reply window for both actuators	bus traffic per cycle at 1 Mbps (~10% load)	to declare a silent actuator faulted (3 cycles)	controller-side watchdog: motors stop if commands cease

CPU cost is tens of µs per cycle. Jitter, per-axis latency and overruns are logged every cycle and shown on the ground dashboard.

Problems I found and solved

Situation	Reviewing the TVC code before bench testing, I found a path where every command could be pinned to a single actuator length.
Task	Remove the hazard, even though unit tests passed, because it depended on link order.
Action	The cause was C++ static-initialization order. A geometry constant was computed at runtime, but the supervisor used it in its own global constructor, so on some link orders it read 0 and the length clamp collapsed. I made the constant `constexpr` (compile-time square root) with a `static_assert` on its value.
Result	An entire class of build-order bugs is now impossible, and a geometry change that breaks it fails at compile time instead of on hardware.

Situation	Control logic runs on one 1.5 ms work queue; waiting there for CAN replies (up to 5 ms) would stall it, and the two raced on shared state.
Task	Make the 50 Hz TVC loop deterministic without slowing the 1.5 ms controller.
Action	Moved CAN onto a dedicated thread that alone owns the bus, paced by an absolute kernel timer, with a mutex-guarded hand-off. Moved logging out of the critical section (it could stall the controller) and fixed an undefined NaN-to-int conversion in the bench log.
Result	The controller tick never waits on CAN, and command-to-reply latency stays under 5 ms at a stable 50 Hz, with jitter and overruns measured every cycle instead of hidden.

Next steps (before vehicle integration)

- Enable the 5 Mbps data phase after scope-checking eye quality on the full-length vehicle harness
- Confirm yaw direction and controller ID, then resolve the yaw geometry discrepancy with the mechanical team
- Set an explicit torque limit from the actuator's safe force rating
- Tune slew and tracking limits from measured step response, then run hardware-in-the-loop with GNC

What I learned

- **The electrical layer comes first.** A missing terminator or a floating pin looks like a "software bug" until you put a scope on it.
- **Fail safe by design, in layers:** firmware interlocks, a controller-side watchdog, and a mechanical home reference each cover the others' failures.
- **Refuse to guess.** When data disagreed, I made the system refuse to run and escalated the question, instead of picking a number.